1. INTRODUCTION

The exponential growth of digital content across documents, images, and videos has transformed how information is created, stored, and consumed. Educational institutions, enterprises, and technical domains increasingly rely on large collections of heterogeneous data for knowledge sharing and decision-making. However, accessing precise information from such vast and multimodal sources remains a significant challenge. Users are often required to manually browse lengthy documents, interpret diagrams, or watch videos to locate specific answers, which is time-consuming and inefficient. This problem highlights the need for intelligent systems capable of understanding and retrieving information directly from diverse user-provided content.

Conventional chatbot systems, though powerful, primarily depend on pretrained large language models with static internal knowledge. These systems are designed to generate responses based on patterns learned during training rather than on the specific documents or materials provided by users at runtime. As a result, they struggle to answer queries grounded in external content and may produce hallucinated or unverifiable responses. This limitation reduces their reliability in domains where factual accuracy and source traceability are critical, such as education, research, enterprise knowledge management, and technical documentation analysis.

To address these shortcomings, this project proposes an Intelligent Question Answering Chatbot based on a Multimodal Retrieval-Augmented Generation (RAG) framework. The core idea of Retrieval-Augmented Generation is to enhance language model responses by retrieving relevant contextual information from an external knowledge base before generating an answer. By integrating retrieval mechanisms with generative capabilities, the system ensures that responses are not only fluent but also factually grounded in user-provided data.

A key innovation of this project is its ability to operate on multimodal data sources. Textual information from native PDF documents is extracted using document parsing techniques. For scanned pages, images, and embedded diagrams, Optical Character Recognition (OCR) is employed to convert visual content into machine-readable text. Videos containing spoken explanations are processed using speech-to-text transcription methods, enabling the system to utilize audio information as part of its knowledge base. Videos without meaningful speech are intelligently filtered using metadata handling to avoid unnecessary processing. This

comprehensive extraction pipeline allows the chatbot to build a unified textual representation from multiple modalities.

The extracted content is then segmented into meaningful semantic chunks to preserve context and improve retrieval precision. These chunks are converted into high-dimensional vector embeddings using pretrained embedding models and stored in a vector database along with contextual metadata such as source references and positional information. This vectorized representation enables efficient similarity-based retrieval when a user submits a query.

During interaction, user queries undergo lightweight normalization and are transformed into embeddings. The system performs a similarity search in the vector database to identify the most relevant content segments related to the query. These retrieved contexts are then passed to a Large Language Model, which generates responses strictly based on the retrieved information. This grounding mechanism significantly reduces hallucinations and ensures that answers are accurate, context-aware, and traceable to their original sources.

By combining multimodal data extraction, vector-based retrieval, and language model generation, the proposed chatbot offers a scalable and reliable solution for conversational information access. It minimizes the need for manual document exploration, enhances knowledge accessibility, and supports informed decision-making across various real-world applications. This approach demonstrates how multimodal Retrieval-Augmented Generation can transform traditional chatbot systems into intelligent assistants capable of reasoning over user-specific knowledge repositories.

2. SUPPORTING LITERATURE

Title	Methodology	Summary
MDKAG: Retrieval-Augmented Educational QA Powered by a Multimodal Disciplinary Knowledge Graph	OCR for documents, text–image extraction from slides, ASR-based transcription for videos, graph-based retrieval	The study shows that integrating multimodal knowledge graphs with RAG improves answer accuracy and reduces hallucinations compared to traditional RAG systems. Experimental results demonstrate better retrieval relevance and reliable question answering across different educational subjects, making the approach scalable and effective for educational applications.
Multimodal retrieval-augmented generation framework for visually rich knowledge in the architecture domain	CLIP, VisualBERT, and Doubao embedding models for multimodal retrieval; Qwen2.5-VL for table parsing; Qwen3 MLLM for multimodal answer generation	Proposes a multimodal RAG framework that unifies text, images, and tables for accurate, traceable knowledge retrieval and generation in architectural design tasks
Knowledge graph-extended retrieval augmented generation for question answering	KG-based multi-hop retrieval, and similarity-based triple selection to generate explainable answers without model training	training-free question answering framework that integrates Large Language Models (LLMs) with Knowledge Graphs (KGs) to improve accuracy and explainability.

Table 1.1 Literature Review

2.2 Literature Review summary

Recent research in Retrieval-Augmented Generation (RAG) shows a clear shift from text-only retrieval toward structured and multimodal knowledge integration to reduce hallucinations and improve answer reliability in question answering systems. The three papers reviewed collectively demonstrate how RAG systems evolve when they incorporate knowledge graphs, multimodal document understanding, and reasoning-aware retrieval strategies.

The first work, MDKAG: Retrieval-Augmented Educational QA Powered by a Multimodal Disciplinary Knowledge Graph by Xu Zhao, Guozhong Wang, and Yufei Lu, addresses limitations of traditional RAG in educational environments. The authors observe that educational content exists in textbooks, slides, and lecture videos, but standard RAG treats this data as flat text chunks. They propose constructing a multimodal disciplinary knowledge graph from OCR-processed documents, PPT slides, and video transcripts. Retrieval is performed through entity relationships in the graph rather than pure embedding similarity, and a post-generation verification mechanism checks semantic similarity and entity coverage to reduce hallucinations. This work highlights that knowledge organization and verification are as important as retrieval.

The second work, Multimodal Retrieval-Augmented Generation framework for visually rich knowledge in the architecture domain by Xianchuan Meng and Ziyu Tong, focuses on challenges in domains where information is highly visual, such as architecture. The authors argue that converting tables, diagrams, and drawings into plain text embeddings destroys structural meaning. Their M-RAG framework decomposes tables into both semantic text and rendered images, embeds text and images into a shared semantic space, and uses a Multimodal LLM to interpret retrieved visual content before answer generation. This paper emphasizes that multimodal embedding alignment and visual reasoning are essential for accurate retrieval in visually rich documents.

The third paper, Knowledge graph-extended retrieval augmented generation for question answering by Jasper Linders and Jakub M. Tomczak, explores how Knowledge Graphs can be combined with RAG without requiring training. Building on earlier systems like KAPING, the authors introduce a question decomposition module that uses Chain-of-Thought prompting to split complex queries into sub-questions. This improves multi-hop triple retrieval and produces an explicit reasoning chain, making answers more explainable. The

study demonstrates that reasoning-aware retrieval significantly improves performance for complex queries.

Across these works, a common theme emerges: traditional RAG fails when knowledge is multimodal, structured, or requires multi-step reasoning. The solutions proposed in these papers extend RAG in three complementary directions:

- Knowledge Graph integration for structured, entity-aware retrieval and explainability (KG-RAG, MDKAG)
- Multimodal data handling for tables, diagrams, and images (M-RAG)
- Question decomposition and reasoning chains to improve multi-hop retrieval (KG-RAG)
- Answer verification mechanisms to reduce hallucinations (MDKAG)

Together, these studies form a strong foundation for building advanced RAG systems capable of operating on heterogeneous user-provided data while maintaining answer accuracy, contextual relevance, and traceability.

2.3 Findings and Proposals

The study of existing chatbot and Retrieval-Augmented Generation systems shows that traditional chatbots relying solely on pretrained language models are unreliable when answering questions from user-provided documents, as they often generate hallucinated or unverifiable responses. Even conventional RAG systems, although improving factual grounding through retrieval, are largely limited to text-only processing and struggle with real-world data that includes scanned PDFs, diagrams, tables, images, and videos. Additionally, retrieval based purely on vector similarity can return contextually irrelevant content if semantic segmentation and metadata are not properly preserved. These observations indicate that reliable document-centric question answering requires multimodal data extraction, semantic indexing, accurate retrieval, and response generation strictly grounded in retrieved content.

Based on these findings, this project proposes an intelligent multimodal Retrieval-Augmented Generation chatbot for answering questions from user-uploaded documents and videos. The system extracts text from native PDFs using PyMuPDF, applies OCR for scanned documents and diagrams, and converts spoken content in videos into text using Whisper, ensuring all information is transformed into a unified textual form. The processed content is semantically

segmented, embedded using the all-MiniLM-L6-v2 model, and stored in a Chroma vector database for efficient similarity-based retrieval. When a user submits a query, relevant content is retrieved and passed to the LLaMA 3 language model, which generates responses strictly based on this context. This approach reduces hallucinations, improves answer accuracy, and provides scalable, reliable conversational access to heterogeneous data sources.

3. Problem Definition

With the rapid growth of digital content, large volumes of information are now stored in the form of PDF documents, scanned files, images, diagrams, presentations, and videos. In educational institutions, enterprises, and technical domains, users frequently need to extract specific information from such document collections. However, locating precise answers within lengthy and heterogeneous files is a time-consuming and inefficient process that often requires manual reading, searching, and interpretation.

Although modern chatbots powered by Large Language Models appear capable of answering questions conversationally, they suffer from a fundamental limitation: they rely on pretrained internal knowledge rather than the user's actual documents. As a result, when users ask questions related to their own files, these chatbots tend to produce hallucinated, generic, or unverifiable responses. Even basic Retrieval-Augmented Generation systems improve this only partially, as they primarily work with plain text and fail to handle information embedded in scanned pages, diagrams, tables, and videos.

In real-world scenarios, important knowledge is frequently present in non-textual formats. Diagrams in technical manuals, scanned academic notes, tables in reports, and spoken explanations in videos contain critical information that traditional text-based retrieval systems cannot process effectively. This creates a gap between how information is stored and how intelligent systems attempt to retrieve it.

Therefore, there is a strong need for a system that can:

- Extract meaningful information from multimodal sources such as PDFs, images, and videos,
- Organize this information in a retrievable form,
- Retrieve the most relevant context for a user's query, and
- Generate answers that are strictly grounded in the user-provided content, ensuring accuracy and traceability.

4. Identify and Analyse the tools/Models

PyMuPDF for Text Extraction

PyMuPDF is a Python binding for the MuPDF engine, designed for fast and precise reading of digitally created (machine-readable) PDFs. It is not an AI model; it is a high-performance PDF parsing library that exposes the internal structure of a PDF page—text blocks, lines, words, coordinates, fonts, and metadata.

In a multimodal RAG system, the quality of retrieval depends heavily on how accurately document text is extracted before embedding. Many PDFs (reports, notes, manuals) are digitally generated and already contain structured text. If this text is extracted naively (e.g., plain copy), the reading order, columns, headers, and positional context are lost. PyMuPDF preserves this structure, which directly improves:

- Semantic chunking quality
- Embedding relevance
- Retrieval accuracy from the vector database

PyMuPDF serves as the foundation of the document understanding pipeline for native PDFs. By preserving layout, structure, and reading order, it enables high-quality semantic chunking and embedding, which are critical for accurate retrieval and reliable answer generation in the multimodal RAG chatbot.

Optical Character Recognition (OCR) for Scanned Documents

Optical Character Recognition (OCR) is a computer vision + pattern recognition pipeline that converts text present inside images (scanned PDFs, photos, diagrams, screenshots) into machine-readable digital text. In your multimodal RAG chatbot, OCR is essential because a large portion of real-world documents are not digitally typed PDFs. They are:

- Scanned class notes
- Printed documents saved as images
- Diagrams with labels
- Tables captured as images
- Old reports and handwritten annotations (sometimes)

Without OCR, all this content is invisible to embeddings, vector search, and the LLM. OCR is what unlocks this hidden knowledge and brings it into the same retrieval pipeline as normal text.

How an OCR System Works Internally

A typical OCR engine (e.g., Tesseract OCR) follows a multi-stage pipeline:

1. Image Preprocessing
    - Convert to grayscale
    - Noise removal (denoising)
    - Binarization (black–white thresholding)
    - Skew correction (straightening tilted scans)
    - Contrast enhancement
2. Text Detection & Segmentation
    - Detect text regions in the image
    - Segment into blocks → lines → words → characters
    - Draw bounding boxes around detected text areas
3. Character Recognition
    - Pattern matching or deep learning model recognizes characters
    - Converts pixel shapes into letters and numbers
4. Post-processing (Language Correction
    - Dictionary-based correction
    - Language model correction (fixes “teh” → “the”)
    - Reconstruct proper sentences from recognized words

Why OCR is Critical for RAG Pipeline

In a multimodal RAG system, embeddings can only be created from text. If a page is scanned:

- Embedding model sees nothing
- Vector DB stores nothing
- LLM cannot answer from that content

OCR converts this non-readable content into retrievable knowledge.

Whisper for Speech-to-Text Transcription

Whisper is a transformer-based Automatic Speech Recognition (ASR) model designed to convert spoken audio into accurate text across multiple languages and accents. It is trained on a very large, diverse audio dataset and is robust to background noise, varied speaking styles, and domain vocabulary. In your multimodal RAG chatbot, Whisper is the component that converts knowledge hidden inside videos (lectures, explanations, demos) into searchable text that can be embedded, stored, and retrieved just like document content.

Why Speech-to-Text is Essential in our System

A large amount of real knowledge today exists in:

- Lecture recordings
- Tutorial videos
- Meeting recordings
- Explanatory demonstrations

Without speech-to-text, this knowledge is invisible to embeddings and vector search. The LLM cannot access it. Whisper transforms this audio knowledge into text so it becomes part of the same retrieval pipeline as PDFs and OCR output.

Architecture of Whisper

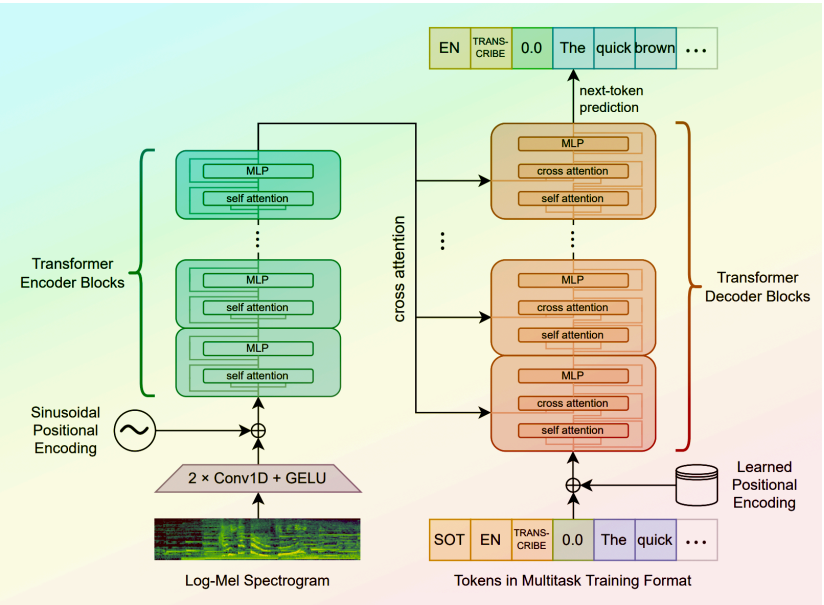


Figure 1.1: Architecture of Whisper

Whisper follows an encoder–decoder transformer architecture:

1. Audio Preprocessing
    - Input audio is converted into a log-Mel spectrogram (a visual representation of sound frequencies over time).
    - This turns audio into a format similar to an image that transformers can process.
2. Encoder
    - The encoder processes the spectrogram and learns patterns of speech, tone, pitch, and phonetics.
    - It produces a compressed representation of the spoken content.
3. Decoder
    - The decoder predicts the corresponding text tokens step by step.
    - It uses language understanding to correct mistakes and produce grammatically meaningful sentences.
4. Timestamp Alignment
    - Whisper can produce timestamps for each sentence/segment.
    - This allows mapping text back to exact video time locations.

all-MiniLM-L6-v2 for Semantic Text Embeddings

The all-MiniLM-L6-v2 model is a sentence embedding model from the Sentence Transformers family, specifically designed to convert text into dense numerical vector representations that capture semantic meaning rather than surface-level word similarity. In this project, it plays a central role by transforming all extracted textual content whether from PDFs, OCR output, or video transcripts into embeddings that can be efficiently stored, searched, and compared within a vector database.

Unlike traditional keyword-based or lexical search methods, all-MiniLM-L6-v2 enables semantic retrieval, meaning that conceptually similar text segments can be retrieved even if they do not share exact keywords. This capability is essential for Retrieval-Augmented Generation systems, where the quality of retrieval directly determines the accuracy of the generated responses.

What Problem Embedding Models Solve

In document-centric QA systems, raw text cannot be directly compared in a meaningful way. Consider the following:

- “What is retrieval-augmented generation?”
- “Explain how RAG improves LLM accuracy”

Although these sentences use different words, they express the same intent. Keyword matching would treat them as different, but an embedding model maps them to nearby points in vector space, enabling accurate retrieval.

The all-MiniLM-L6-v2 model solves this problem by encoding semantic meaning, context, and intent into a fixed-length vector, making similarity computation mathematically feasible.

Internal Architecture and Design

all-MiniLM-L6-v2 is based on the MiniLM architecture, which is a compressed transformer model derived from larger language models using knowledge distillation. Key architectural characteristics include:

- 6 transformer layers, making it lightweight and fast
- 384-dimensional output embeddings
- Transformer-based self-attention mechanism
- Optimized for sentence-level semantic similarity

The model is trained using contrastive learning, where semantically similar sentence pairs are pulled closer in vector space, while unrelated pairs are pushed farther apart. This training objective is what allows the model to capture meaning rather than syntax.

1. Input Text Processing
    - Each semantic chunk (from PDFs, OCR text, or video transcripts) is tokenized into subword units using a transformer tokenizer.
2. Contextual Encoding
    - Tokens pass through multiple transformer layers where self-attention captures relationships between words, phrases, and sentence-level context.

3. Sentence Representation
    - Token-level representations are pooled (mean pooling) to produce a single dense vector representing the entire chunk.
4. Vector Normalization
    - The output vector is normalized, enabling efficient cosine similarity computation during retrieval.

LLaMA 3 8B for Answer Generation

LLaMA 3 (8B parameters) is a state-of-the-art Large Language Model developed by Meta and is used in this project as the core generative component responsible for producing final user-facing answers. Unlike earlier chatbot systems that rely solely on pretrained internal knowledge, LLaMA 3 is integrated into a Retrieval-Augmented Generation (RAG) pipeline, where its output is explicitly grounded in externally retrieved information. This design choice ensures that the model functions not as a standalone conversational agent but as a controlled reasoning engine that generates responses strictly based on the contextual evidence supplied to it.

LLaMA 3 is built on a decoder-only transformer architecture, which has become the dominant paradigm for modern generative language models. The transformer design enables the model to capture long-range dependencies in text using self-attention mechanisms rather than sequential recurrence. The 8-billion-parameter variant represents a balanced configuration that provides strong language understanding and generation capabilities while remaining computationally feasible for deployment in applied systems. This balance is particularly important in document-centric applications, where inference efficiency and response latency are critical factors.

Internally, the model processes input text by first converting tokens into dense vector representations through an embedding layer. These embeddings pass through multiple stacked transformer blocks, each consisting of multi-head self-attention layers and feed-forward neural networks. The self-attention mechanism allows the model to dynamically focus on different parts of the input context, assigning higher importance to relevant tokens while generating each output token. This capability is especially important in RAG-based

systems, where the prompt includes retrieved document excerpts that must be accurately referenced during answer generation.

In the context of this project, LLaMA 3 does not operate on raw user queries alone. Instead, the system constructs a structured prompt that includes the user's question along with the most relevant contextual chunks retrieved from the vector database. These chunks originate from PDFs, OCR-processed scanned documents, and Whisper-transcribed video content. By supplying this retrieved context directly to the model, the system constrains LLaMA 3 to base its responses on verifiable external information, significantly reducing hallucination and improving factual correctness.

A key strength of LLaMA 3 lies in its strong instruction-following behavior and contextual reasoning ability. When prompted appropriately, the model can synthesize information across multiple retrieved chunks, identify relationships between concepts, and produce coherent, well-structured answers. This is particularly useful when user questions require summarization, explanation, or comparison rather than direct fact lookup. The model's capacity to integrate information across multiple context passages allows it to function as a reasoning layer on top of the retrieval system.

Another important aspect of LLaMA 3 is its scalability and controllability within a RAG architecture. Since the model does not require fine-tuning for each new document collection, it can be reused across different datasets simply by updating the retrieval corpus. This aligns well with the project's objective of supporting user-specific documents, where the knowledge base may change frequently. The separation between retrieval and generation ensures that updates to the document set do not require retraining the language model itself.

From a reliability perspective, the use of LLaMA 3 in a grounded generation setting directly addresses one of the most critical weaknesses of large language models: hallucination. In traditional chatbot systems, the model may generate plausible-sounding but incorrect answers when information is missing or ambiguous. In contrast, this project restricts the model's input to retrieved context, effectively limiting its output space to information that exists within the user-provided data. As a result, answers are not only more accurate but also traceable to their original sources.

The choice of the 8B parameter variant is also significant. Larger models may offer marginal gains in fluency, but they come with increased computational cost and deployment complexity. The 8B model provides a practical compromise by delivering high-quality language generation while remaining suitable for real-time inference in an applied system. This makes it particularly appropriate for academic and enterprise use cases, where resource constraints must be considered alongside performance.

In summary, LLaMA 3 (8B) serves as the final reasoning and response generation engine in the proposed multimodal RAG chatbot. By operating within a retrieval-augmented framework, the model combines the expressive power of large language models with the reliability of document-grounded knowledge access. Its transformer-based architecture, strong contextual reasoning abilities, and efficient parameter scale make it well-suited for generating accurate, coherent, and context-aware answers from heterogeneous user-provided data. This integration transforms LLaMA 3 from a generic conversational model into a dependable component of a document-centric question answering system.

Architecture of LLaMA 3

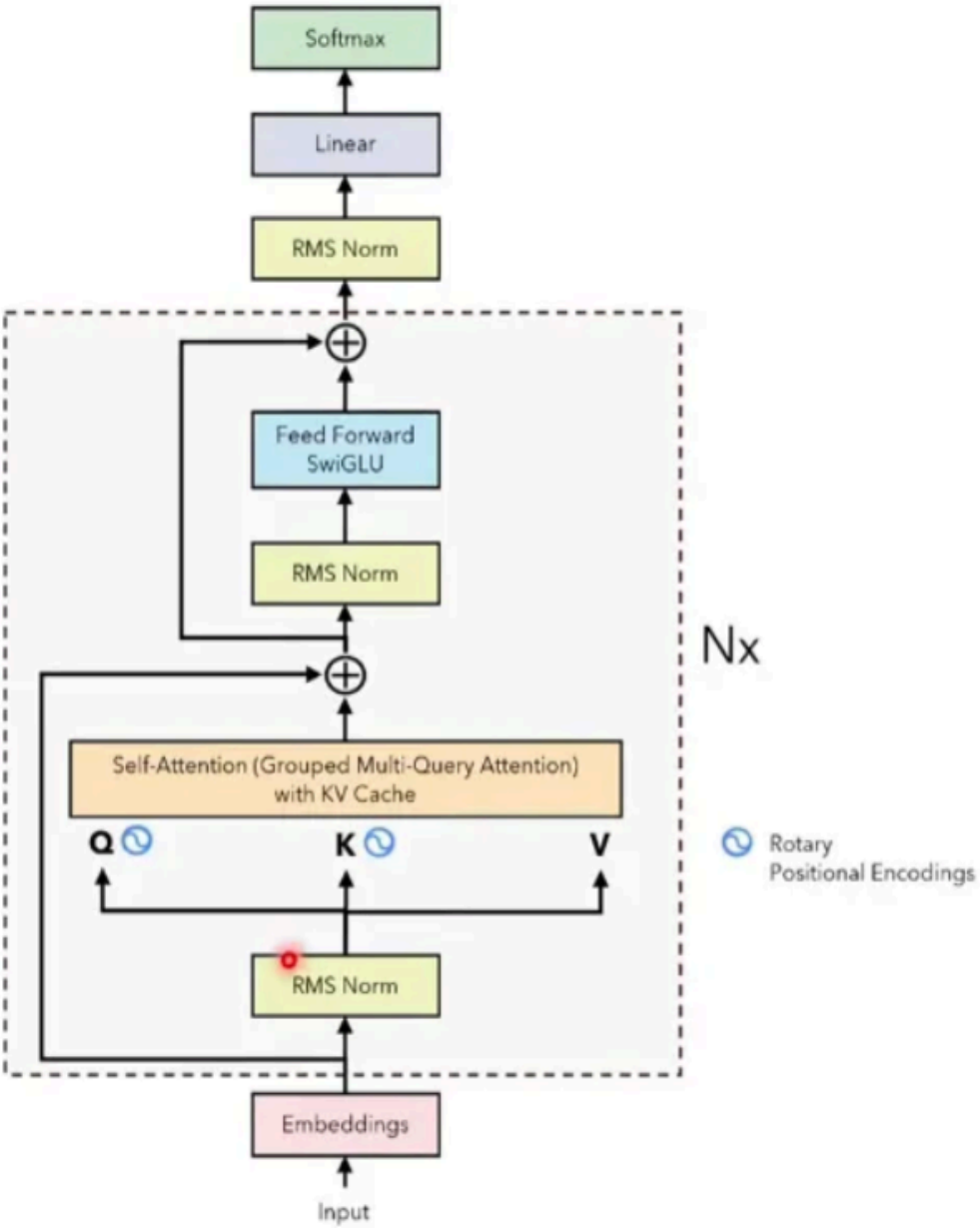


Figure 1.2 Architecture of LLaMA 3

LLaMA 3 follows a decoder-only Transformer architecture, optimized for efficient large-scale text generation. The model consists of an input embedding layer followed by N repeated transformer blocks, and finally a projection layer that produces token probabilities.

Each component in the architecture is designed to improve training stability, inference efficiency, and contextual reasoning.

- Input Embeddings

The architecture begins with token embeddings, where each input token (generated using a tokenizer) is mapped to a dense vector representation. These embeddings encode semantic meaning and serve as the initial numerical representation of the input text.
Unlike traditional RNN-based models, LLaMA 3 processes the entire input sequence in parallel using these embeddings.

- RMS Normalization (Pre-Norm Design)

Before entering both the attention and feed-forward sublayers, RMSNorm (Root Mean Square Normalization) is applied.
Purpose of RMSNorm:

➔ Stabilizes training for very deep transformer networks
➔ Reduces computational overhead compared to LayerNorm
➔ Improves numerical stability during inference

LLaMA 3 uses a pre-norm architecture, meaning normalization is applied *before* each major operation rather than after. This helps prevent gradient instability as depth increases.

- Self-Attention with Grouped Multi-Query Attention (GQA)

The core reasoning component of each transformer block is the Self-Attention layer, implemented using Grouped Multi-Query Attention with KV Cache.
Self-Attention
Self-attention allows each token to attend to all previous tokens in the sequence, enabling the model to capture long-range dependencies and contextual relationships.
Each token produces:

➔ Query (Q) – what the token is looking for
➔ Key (K) – what the token offers
➔ Value (V) – the information passed forward

Attention weights are computed between Q and K, then applied to V.

Grouped Multi-Query Attention (GQA)

Instead of using separate Key and Value matrices for every attention head (as in standard Multi-Head Attention), LLaMA 3 shares K and V across groups of heads.

Why GQA is used:

- ➔ Significantly reduces memory usage
- ➔ Improves inference speed
- ➔ Maintains high attention quality

This design is especially important for large models like LLaMA 3, enabling scalable deployment without sacrificing performance.

KV Cache

During text generation, previously computed Key and Value tensors are stored in a KV cache. This avoids recomputation for past tokens and allows fast autoregressive decoding.

- Rotary Positional Embeddings (RoPE)

Instead of absolute positional embeddings, LLaMA 3 uses Rotary Positional Embeddings applied directly to the Query and Key vectors.

Advantages of RoPE:

- ➔ Encodes relative positional information naturally
- ➔ Improves performance on long contexts
- ➔ Enables extrapolation to longer sequences than seen during training

RoPE allows the model to understand order and distance between tokens without adding explicit position vectors.

- Residual Connections

After the self-attention output, a residual (skip) connection adds the original input back to the output.

- Feed-Forward Network with SwiGLU

Following attention, the output passes through a Feed-Forward Neural Network (FFN) implemented using SwiGLU activation.

SwiGLU (Swish-Gated Linear Unit)

SwiGLU replaces traditional ReLU or GELU activations.

Benefits:

- ➔ Improves expressiveness
- ➔ Enables gated information flow
- ➔ Enhances language modeling performance
- ➔ The FFN expands the hidden dimension, applies SwiGLU activation, and then projects it back to the original size.

- Second RMSNorm and Residual Connection

Another RMSNorm is applied before the FFN, followed by a residual connection after it.

This completes one transformer block.

- Repeated Transformer Blocks (N×)

The dashed box labeled N× indicates that this entire block—attention + FFN—is repeated multiple times.

- Final RMSNorm, Linear Layer, and Softmax

After passing through all transformer blocks:

- ➔ Final RMSNorm stabilizes output representations
- ➔ Linear layer projects hidden states to vocabulary size
- ➔ Softmax converts logits into probability distributions over tokens

The token with the highest probability is selected during generation.

5. Project Pipeline

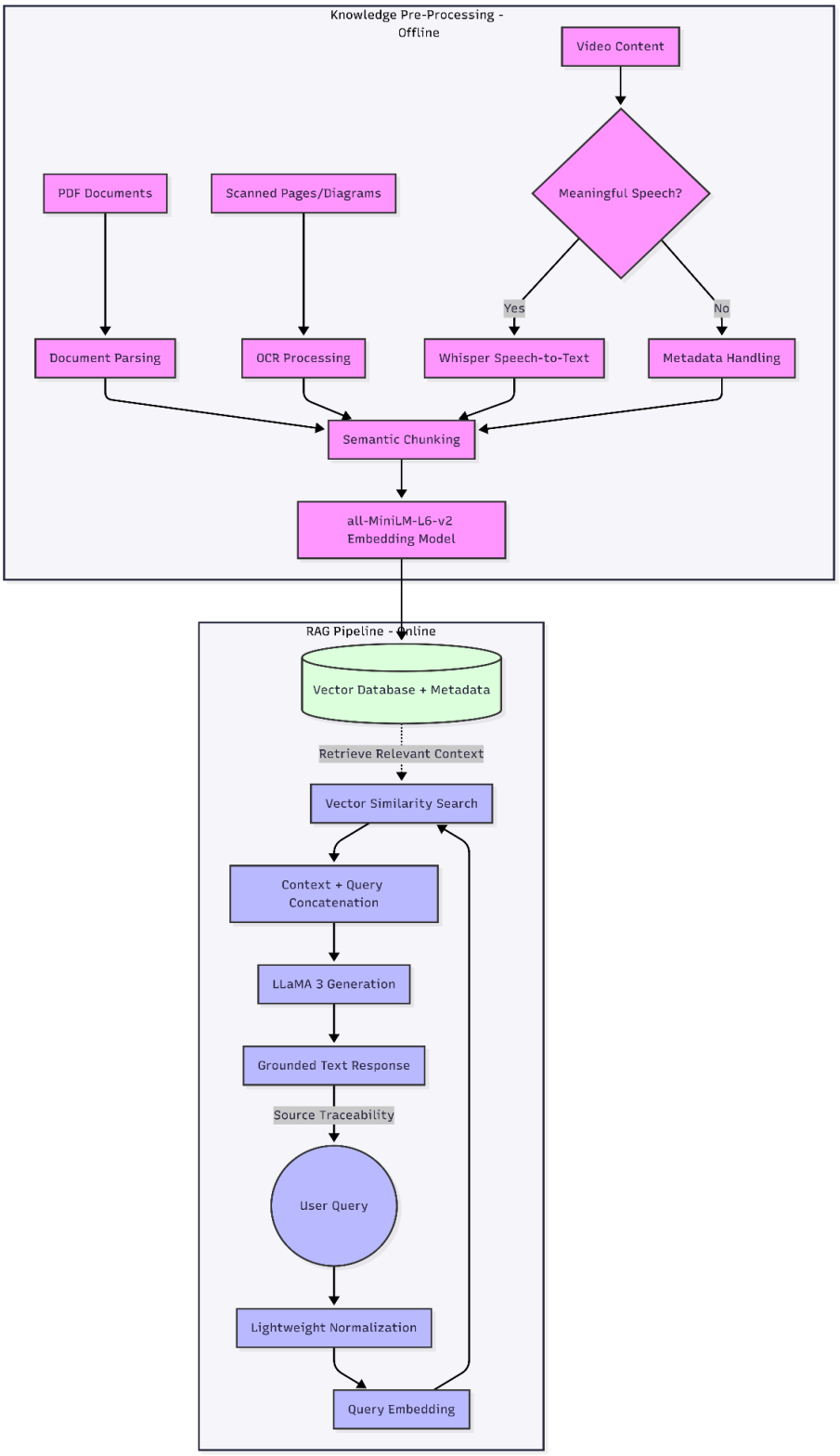


Figure 1.3: Project pipeline

1. User Interaction and Data Ingestion:

The pipeline begins with user interaction, where the user uploads documents such as PDF files, images, or videos and submits natural language questions through the chatbot interface. The uploaded files act as the dynamic knowledge source for the system, enabling personalized and document-centric question answering.

2. Multimodal Content Processing

To handle heterogeneous input formats, the system converts all modalities into a unified textual representation:

- PDF Processing:

Native PDFs are parsed to extract textual content page-wise, while scanned PDFs are processed using Optical Character Recognition (OCR).

- Image Processing:

Images containing textual information, diagrams, or figures are processed using OCR techniques to extract embedded text.

- Video Processing:

For videos containing spoken content, the audio stream is extracted and converted into text using automatic speech recognition (ASR). Videos without informative speech are handled using metadata and skipped during transcription.

3. Text Preprocessing and Structure-Aware Chunking

The extracted text is cleaned and segmented into smaller, meaningful units known as chunks. Instead of naïve fixed-length chunking, the system adopts a structure-aware chunking strategy, where document elements such as titles, section headings, content blocks, and footers are identified and labeled using metadata. This improves retrieval accuracy, especially for structural queries such as titles or section-specific questions.

4. Vectorization and Knowledge Indexing

Each text chunk is transformed into a dense vector representation using a pretrained sentence embedding model. These embeddings capture the semantic meaning of the text. The vectors, along with associated metadata (page number, document source, content type), are stored in a vector database (ChromaDB). This indexed vector store forms the searchable knowledge base of the chatbot.

5. Query Processing and Retrieval

When a user submits a query, the question is preprocessed and converted into an embedding using the same embedding model. The vector database performs a similarity search to retrieve the top-k most relevant chunks based on semantic similarity and metadata filters. This retrieval step ensures that only contextually relevant information is selected for answer generation.

6. Context Augmentation and Answer Generation

The retrieved text chunks are aggregated and combined with the user query to form an augmented context. This context is passed to a Large Language Model (LLM), which generates a response conditioned strictly on the retrieved evidence. By grounding the generation process in retrieved content, the system minimizes hallucinations and improves factual consistency.

7. Response Delivery and Explainability

The generated answer is presented to the user along with optional references such as source document names or page numbers. This enhances transparency and allows users to verify the origin of the information. The chatbot supports conversational interaction, enabling follow-up questions within the same context.

8. System Evaluation and Validation

The system is evaluated using retrieval and answer quality metrics such as precision at k, semantic similarity, exact match, and qualitative user assessment. These evaluations measure the effectiveness of retrieval accuracy, contextual relevance, and response reliability.

6. SYSTEM DESIGN

The proposed system is designed as a Multimodal Retrieval-Augmented Generation (RAG) pipeline that enables users to ask questions about uploaded documents, images, and videos.

The system extracts information from different media types, converts them into semantic embeddings, and retrieves relevant context to generate accurate answers using a large language model.

The system architecture consists of the following components:

- Document and Media Ingestion
- Text Extraction and Processing
- Embedding Generation
- Vector Database Storage
- Hybrid Retrieval
- Answer Generation using LLM

6.1 Model Building

The model building phase focuses on constructing a multimodal information retrieval and question answering system using a Retrieval-Augmented Generation approach.

Unlike traditional machine learning models, the system does not require supervised training. Instead, it builds an information retrieval pipeline that converts multimedia data into searchable embeddings and uses a large language model to generate answers.

6.1.1 Python Libraries Used

The proposed system is implemented in Python and utilizes several libraries for document processing, machine learning, multimedia analysis, and retrieval operations. These libraries enable the system to process multimodal data such as text, images, and videos and perform retrieval-augmented question answering.

1. **Unstructured**

The Unstructured library is used in this project for structured document parsing and text extraction from PDF files. It is a specialized library designed to process complex documents and identify different structural elements such as titles, paragraphs, lists, and tables.

In this project, the partition_pdf() function from the Unstructured library is used to convert PDF documents into structured text components that can be further processed by the system.

```
from unstructured.partition.pdf import partition_pdf
```

Role in the Proposed System

In the proposed multimodal question answering system, the Unstructured library performs the following tasks:

- ➔ PDF Parsing

It reads the uploaded PDF document and identifies different elements within the document.

- ➔ Structured Text Extraction

Instead of extracting raw text, the library categorizes text into elements such as: Title, Section Title, Narrative Text, List Items.

- ➔ Chunk Generation

These extracted elements are converted into text chunks which are used for embedding generation.

- ➔ Metadata Generation

It also provides metadata such as page numbers and element types, which are stored in the vector database and used during retrieval.

2. **Sentence Transformers**

The SentenceTransformers library is used in this project to convert textual information into semantic embeddings. Embeddings are numerical vector representations of text that capture the meaning and contextual relationships between words and sentences.

In the proposed system, the all-MiniLM-L6-v2 model from the SentenceTransformers library is used to generate embeddings for all extracted text chunks.

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
```

Role of SentenceTransformers in the Proposed System

The main objective of the project is to build a Retrieval-Augmented Generation (RAG) system that answers questions based on user-provided documents and multimedia content. To achieve this, the system must be able to understand the semantic meaning of text rather than relying only on exact keyword matching.

SentenceTransformers is used because it enables:

- ➔ Conversion of text into high-dimensional semantic vectors
- ➔ Meaning-based similarity search
- ➔ Efficient comparison between queries and document content

This allows the system to retrieve information that is semantically related to the user query, even if the wording is different.

In the proposed system, the SentenceTransformers library performs the following tasks:

- ➔ Text Embedding Generation

All extracted text chunks from documents, images, and videos are converted into numerical vector representations.

Example:

Text Chunk → Embedding Vector

- ➔ Query Embedding

When a user asks a question, the query is also converted into a vector using the same embedding model.

User Query → Query Embedding

- ➔ Semantic Similarity Matching

The query embedding is compared with stored document embeddings using vector similarity search in the vector database.

This enables the system to find the most relevant context for answering the question.

3. **PDF2Image Library**

The pdf2image library converts PDF pages into image format.

```
from pdf2image import convert_from_path
```

Role in the Project

The library converts each PDF page into an image so that the Vision Language Model can analyze the content.

4. **Pillow (PIL)**

The Pillow library is used for image processing tasks.

```
from PIL import Image
```

Role in the Project

Pillow is used to:

- ➔ Open extracted images
- ➔ Check image dimensions
- ➔ Filter small images or icons
- ➔ Prepare images for analysis

5. **PyMuPDF (fitz)**

The PyMuPDF library is used to extract embedded images from PDF documents.

```
import fitz
```

Role in the Project

PyMuPDF is used to:

- ➔ Access PDF pages
- ➔ Identify embedded images
- ➔ Extract images from the document

6. **Base64 Library**

The Base64 module is used to encode image files before sending them to the vision language model.

```
import base64
```

Role in the Project

The library converts image data into Base64 strings that can be embedded in API requests.

7. **FFmpeg**

FFmpeg is a powerful multimedia processing tool used to manipulate video and audio files.

Role in the Project

Video files must be processed to extract both audio and visual frames.

FFmpeg is used to:

- ➔ Extract audio tracks from videos
- ➔ Convert audio to a format compatible with Whisper
- ➔ Extract video frames for vision analysis

8. Vision Language Model (VLM)

In the proposed system, a Vision Language Model (VLM) is used to analyze images and extract meaningful textual descriptions from them. The model used in this project is Llama-4 Scout, which can process both visual and textual inputs.

Comparison Between OCR and Vision Language Model for Image Understanding

During the development of the system, two different approaches were tested for extracting information from images and diagrams contained in the document:

- ➔ Optical Character Recognition (OCR)
- ➔ Vision Language Model (VLM)

The objective was to determine which method provides better contextual information for the Retrieval-Augmented Generation (RAG) pipeline.

OCR-Based Extraction

Initially, the system used OCR techniques to extract text from images and scanned PDF pages.

OCR focuses on detecting visible characters in an image and converting them into machine-readable text. While this approach works well for simple text images, it struggles with complex visual structures such as diagrams and flowcharts.

The ocr_ingest() function processes input files and extracts textual content. First, the system checks the file type (PDF or image). If the input is a PDF, each page of the document is converted into an image using the pdf2image library. The image is then passed to the Tesseract OCR engine to recognize and extract text.

For image files (.jpg, .jpeg, .png), the image is directly processed using the Python Imaging Library (PIL) and text is extracted using Tesseract.

The extracted text is then divided into smaller chunks using a chunking function. Each chunk is stored with metadata such as a unique ID, the extracted text, the type (ocr_text), and the page number. These chunks are later converted into embeddings and stored in the vector database for retrieval.

However, OCR has several limitations, particularly when processing diagrams, charts, and complex layouts, as it can only extract visible text and cannot interpret visual structures. Due to these limitations, the OCR-based approach was later replaced with a Vision Language Model (VLM), which can better understand multimodal content such as diagrams and generate more accurate textual descriptions.

```
def ocr_ingest(path):
    chunks = []
    idx = 0
    ext = os.path.splitext(path)[1].lower()

    if ext == ".pdf":
        pages = convert_from_path(path, dpi=300)
        for page_num, page_img in enumerate(pages, start=1):
            text = pytesseract.image_to_string(page_img).strip()
            if text:
                pieces = chunk_text(text)
                for piece in pieces:
                    chunks.append({
                        "id": f"ocr_{idx}",
                        "text": piece,
                        "type": "ocr_text",
                        "page": page_num
                    })
                    idx += 1

    elif ext in [".jpg", ".jpeg", ".png"]:  # ✅ same level as if ext == ".pdf"
        img = Image.open(path)
        text = pytesseract.image_to_string(img).strip()
        if text:
            pieces = chunk_text(text)
            for piece in pieces:
                chunks.append({
                    "id": f"ocr_{idx}",
```

```
        for piece in pieces:
            chunks.append({
                "id": f"ocr_{idx}",
                "text": piece,
                "type": "ocr_text",
                "page": 1
            })
            idx += 1

    return chunks
```

Figure 6.1 OCR Ingest

```
data = collection.get()
data
```


```
{'ids': ['ocr_0', 'ocr_1', 'ocr_2'],
 'embeddings': None,
 'documents': ["RAG Pipeline (Online)\n\n \n\n \n\n \
```

Vision Language Model Extraction

To overcome these limitations, the system was upgraded to use a Vision Language Model (VLM) for image understanding.

To improve the extraction of information from images and diagrams, the system uses a Vision Language Model (VLM) instead of traditional OCR. Vision LLMs are capable of understanding both textual and visual content within images, allowing the system to interpret diagrams, charts, and complex visual structures.

The vision_extract() function processes an input image and sends it to the Vision Language Model for analysis. First, the image file is read and converted into Base64 format, which allows it to be transmitted to the model through the API request. The system also determines the correct MIME type (image/png or image/jpeg) based on the image format.

The encoded image is then sent to the LLaMA-4 Vision model using the LLM API. Along with the image, the system includes a prompt instructing the model to extract and describe all textual and visual information present in the image.

The Vision Language Model analyzes the image and generates a detailed textual description of its content. This output may include recognized text, diagram labels, and explanations of visual elements.

The generated text is returned by the function and later processed through text chunking and embedding generation before being stored in the vector database for retrieval.

Using a Vision Language Model significantly improves the system's ability to process multimodal content, particularly diagrams and flowcharts, which are difficult for traditional OCR systems to interpret accurately.

```python
def vision_extract(image_path):
    ext = os.path.splitext(image_path)[1].lower()
    mime = "image/png" if ext == ".png" else "image/jpeg"

    with open(image_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode("utf-8")

    response = client_llm.chat.completions.create(
        model="meta-llama/llama-4-scout-17b-16e-instruct",  # ✅ updated model
        messages=[
            {
                "role": "user",
                "content": [
                    {
                        "type": "image_url",
                        "image_url": {"url": f"data:{mime}; base64,{image_data}"}
                    },
                    {
                        "type": "text",
                        "text": "Extract and describe all text and content from this image in detail."
                    }
                ]
            }
        ]
    )
    return response.choices[0].message.content
```

Figure 6.3 Vision LLM implementation

```
  'ocr_1_5',
  'ocr_1_6',
  'ocr_1_7'],
 'embeddings': None,
 'documents': ['The image presents a flowchart illustrating the RAG Pipeline, which appears to be a multimodal processing system. The pipeline is divided into two main
sections: "RAG Pipeline (Online)" and "Knowledge Pre-Processing (Offline)". \n\n**RAG Pipeline (Online)**\n\n*   **User**: The user initiates the process by providing
input.\n*   **Multimodal Query**: The user\'s input is processed as a multimodal query.',
  ".\n*   **Multimodal Query**: The user's input is processed as a multimodal query.\n*   **Query Normalization**: The multimodal query undergoes normalization.\n*   **Query
Embeddings**: The normalized query is then converted into query embeddings.\n*   **Vector Similarity Search**: The query embeddings are used for vector similarity search.\n*
**Relevant Context**: The search results in relevant conte",
  'imilarity search.\n*   **Relevant Context**: The search results in relevant context.\n*   **Multimodal Embeddings**: The relevant context and query embeddings are combined
to form multimodal embeddings.\n\n**Knowledge Pre-Processing (Offline)**\n\n*   **PDF Files (Text)**: PDF files containing text are processed.\n*   **Scanned Pages (OCR-
Mataa)**: Scanned pages are processed using Optical Character Reco',
  'ed Pages (OCR-Mataa)**: Scanned pages are processed using Optical Character Recognition (OCR) and Mataa.\n*   **Multimodal Content Extraction & Chunking**: The processed
PDF files and scanned pages undergo multimodal content extraction and chunking.\n*   **Video (Speech-to-Text)**: Video content is processed using speech-to-text
technology.\n*   **Grounded Text Response**: The output of the multimoda',
  '-to-text technology.\n*   **Grounded Text Response**: The output of the multimodal content extraction and chunking, as well as the video processing, is used to generate a
grounded text response.\n\n**Additional Components**\n\n*   **Embedding Model**: An embedding model is used to process the multimodal embeddings.\n*   **Multimodal
Embeddings**: The output of the embedding model is multimodal embedding',
  'ltimodal Embeddings**: The output of the embedding model is multimodal embeddings.\n*   **Source Traceability**: The source traceability component is connected to both the
RAG Pipeline (Online) and Knowledge Pre-Processing (Offline) sections.\n\n**Overall Process**\n\nThe RAG Pipeline (Online) section processes user input and generates a text
response, while the Knowledge Pre-Processing (Offline) secti',
  "nd generates a text response, while the Knowledge Pre-Processing (Offline) section prepares the knowledge base for the pipeline. The two sections are connected through the
multimodal embeddings and source traceability components. The flowchart provides a detailed overview of the RAG Pipeline's architecture and functionality.",
  'ality.'],
 'uris': None,
 'included': ['metadatas', 'documents'],
 'data': None,
 'metadatas': [{'type': 'ocr_text', 'page': 1},
  {'type': 'ocr_text', 'page': 1},
  {'page': 1, 'type': 'ocr_text'},
  {'page': 1, 'type': 'ocr_text'},
```


Figure 6.4 Vision LLM result

Observations from Vision LLM Output

The Vision LLM successfully extracted:

- ➔ structured descriptions of diagrams
- ➔ relationships between system components
- ➔ complete explanations of the RAG pipeline diagram

Comparison of OCR and Vision LLM

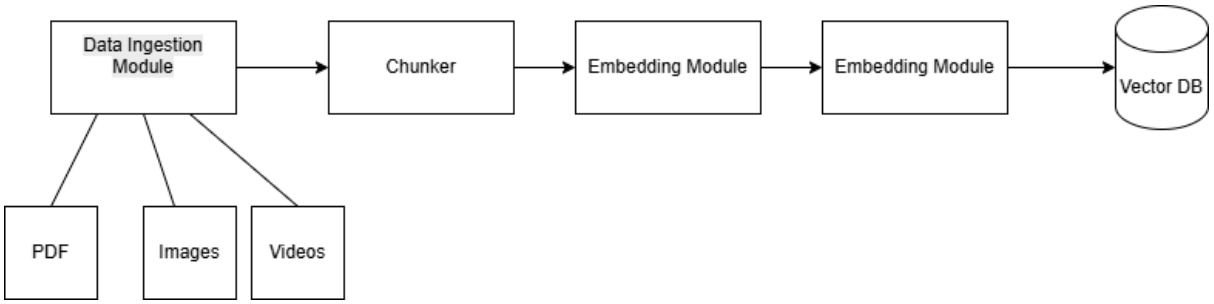
feature	OCR	VisionLLM
Text extraction	YEs	Yes
Diagram understanding	No	Yes

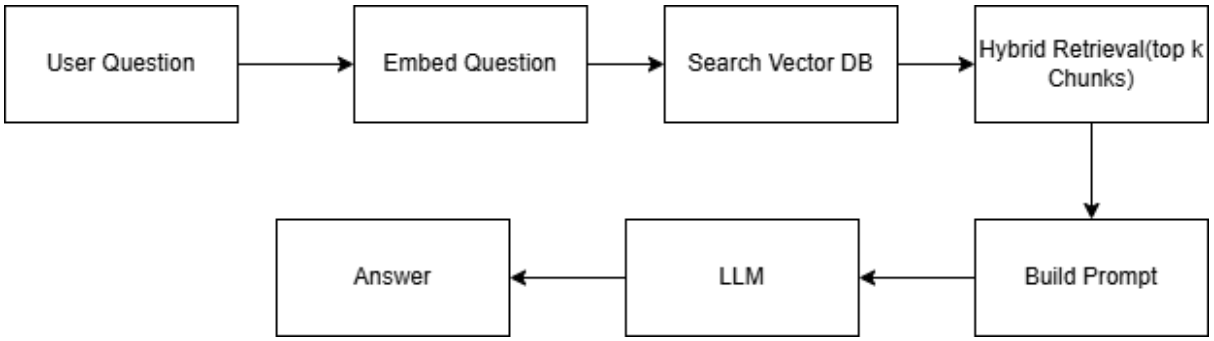
Contextual interpretation	NO	Yes
Handling flowcharts	Poor	Good
Quality of extracted text	Low	High
Suitability for RAG retrieval	Limited	High

6.2 Implementation Code

The implementation of the proposed system consists of three separate processing pipelines designed to handle different types of input data: PDF documents, images, and videos. Each pipeline extracts meaningful textual information which is later converted into embeddings and stored in the vector database for retrieval.

SETUP(done once)





1. Data Ingestion Module

```
def ingest(path):
    ext = os.path.splitext(path)[1].lower()

    if ext in [".jpg", ".jpeg", ".png"]:
        print("Image detected → Vision LLM")
        return ocr_ingest(path)

    if ext in [".mp4", ".mov", ".avi", ".mkv", ".webm"]:  # ✅ new
        print("Video detected → Whisper")
        return video_ingest(path)

    if ext == ".pdf":
        chunks = structured_pdf_ingest(path)
        if len(chunks) < 3:
            print("Scanned PDF → Vision LLM")
            chunks = ocr_ingest(path)
        else:
            print(f"Normal PDF → {len(chunks)} text chunks")

        image_chunks = extract_images_from_pdf(path)
        if image_chunks:
            print(f"Found {len(image_chunks)} embedded image chunks → Vision LLM")
            chunks += image_chunks

        return chunks
```

Figure 6.4 Ingestion layer

The Data Ingestion Module is responsible for collecting input data from different sources and converting it into structured textual chunks that can be used for embedding generation and retrieval in the RAG system. The proposed system supports multimodal inputs, including PDF documents, images, and video files.

The ingestion module first identifies the type of input file and then applies the appropriate processing pipeline to extract meaningful textual information.

The ingestion process begins with detecting the input file type using the file extension. Based on the detected type, the system routes the file to the corresponding processing pipeline.

```python
from unstructured.partition.pdf import partition_pdf
from pdf2image import convert_from_path
from PIL import Image
import fitz
import base64
import os
import io

def vision_extract(image_path):
    ext = os.path.splitext(image_path)[1].lower()
    mime = "image/png" if ext == ".png" else "image/jpeg"

    with open(image_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode("utf-8")

    response = client_llm.chat.completions.create(
        model="meta-llama/llama-4-scout-17b-16e-instruct",  # ✅ updated model
        messages=[
            {
                "role": "user",
                "content": [
                    {
                        "type": "image_url",
                        "image_url": {"url": f"data:{mime}; base64,{image_data}"}
                    },
                    {
                        "type": "text",
```

```python
                    },
                    {
                        "type": "text",
                        "text": "Extract and describe all text and content from this image in detail."
                    }
                ]
            }
        ]
    )
    return response.choices[0].message.content

# —— Chunk Text ——
def chunk_text(text, chunk_size=400, overlap=80):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks


# —— 1. Structured PDF (unstructured) ——
def structured_pdf_ingest(path):
    elements = partition_pdf(path)
    chunks = []
    idx = 0

    for el in elements:
```

```python
    idx = 0

    for el in elements:
        if el.text and el.text.strip():
            page_number = getattr(el.metadata, "page_number", None)
            text = el.text.strip()
            words = text.split()
            refined_type = el.category

            if el.category == "Title":
                if page_number == 1 and len(words) > 5:
                    refined_type = "main_title"
                else:
                    refined_type = "section_title"

            chunks.append({
                "id": str(idx),
                "text": text,
                "type": refined_type,
                "page": page_number
            })
            idx += 1

    return chunks


# —— 2. Scanned PDF + Direct Images → Vision LLM ——
def ocr_ingest(path):
    chunks = []
    ext = os.path.splitext(path)[1].lower()
```

```python
# —— 2. Scanned PDF + Direct Images → Vision LLM ——
def ocr_ingest(path):
    chunks = []
    ext = os.path.splitext(path)[1].lower()

    if ext == ".pdf":
        pages = convert_from_path(path, dpi=300)
        for page_num, page_img in enumerate(pages, start=1):
            temp_path = f"/tmp/page_{page_num}.jpg"
            page_img.save(temp_path)

            text = vision_extract(temp_path).strip()
            if text:
                for idx, piece in enumerate(chunk_text(text)):
                    chunks.append({
                        "id": f"ocr_{page_num}_{idx}",
                        "text": piece,
                        "type": "ocr_text",
                        "page": page_num
                    })

    elif ext in [".jpg", ".jpeg", ".png"]:
        text = vision_extract(path).strip()
        if text:
            for idx, piece in enumerate(chunk_text(text)):
                chunks.append({
                    "id": f"ocr_1_{idx}",
                    "text": piece,
                    "type": "ocr_text",
                    "page": 1
```

```
    return chunks


# —— 3. Embedded Images in PDF → Vision LLM ——
def extract_images_from_pdf(path):
    chunks = []
    doc = fitz.open(path)

    for page_num, page in enumerate(doc, start=1):
        for img in page.get_images(full=True):
            xref = img[0]
            base_image = doc.extract_image(xref)
            img_bytes = base_image["image"]

            # Skip small icons
            pil_img = Image.open(io.BytesIO(img_bytes))
            if pil_img.width < 200 or pil_img.height < 200:
                continue

            temp_path = f"/tmp/embedded_{page_num}.jpg"
            with open(temp_path, "wb") as f:
                f.write(img_bytes)

            text = vision_extract(temp_path).strip()
            if text:
                for idx, piece in enumerate(chunk_text(text)):
                    chunks.append({
                        "id": f"img_ocr_{page_num}_{idx}",
                        "text": piece,
                        "type": "image_ocr"
```

```
                    chunks.append({
                        "id": f"img_ocr_{page_num}_{idx}",
                        "text": piece,
                        "type": "image_ocr",
                        "page": page_num
                    })

    doc.close()
    return chunks

import whisper
import subprocess
import os

# Load whisper model (base is fast, use "medium" for better accuracy)
whisper_model = whisper.load_model("base")

def extract_audio(video_path):
    """Extract audio from video using ffmpeg"""
    audio_path = "/tmp/extracted_audio.wav"

    subprocess.run([
        "ffmpeg", "-i", video_path,
        "-ar", "16000",  # 16kHz sample rate (whisper requirement)
        "-ac", "1",      # mono audio
        "-y",            # overwrite if exists
        audio_path
    ], capture_output=True)

    return audio_path
```

```
        audio_path
    ], capture_output=True)

    return audio_path

def has_audio(video_path):
    """Check if video has audio track"""
    result = subprocess.run([
        "ffprobe", "-i", video_path,
        "-show_streams", "-select_streams", "a",
        "-loglevel", "error"
    ], capture_output=True, text=True)

    return len(result.stdout.strip()) > 0

def extract_frames(video_path, interval=30):
    """Extract one frame every 'interval' seconds"""
    frames_dir = "/tmp/video_frames"

    # ✅ Clear old frames first
    if os.path.exists(frames_dir):
        import shutil
        shutil.rmtree(frames_dir)

    os.makedirs(frames_dir, exist_ok=True)

    subprocess.run([
        "ffmpeg", "-i", video_path,
        "-vf", f"fps=1/{interval}",
        f"{frames_dir}/frame_%04d.jpg",
```

```
        "ffmpeg", "-i", video_path,
        "-vf", f"fps=1/{interval}",
        f"{frames_dir}/frame_%04d.jpg",
        "-y"
    ], capture_output=True)

    return sorted([
        os.path.join(frames_dir, f)
        for f in os.listdir(frames_dir)
        if f.endswith(".jpg")
    ])


def video_ingest(path):
    chunks = []

    # ── Audio → Whisper ──
    if not has_audio(path):
        print("No audio → skipping transcription")
    else:
        print("Extracting audio...")
        audio_path = extract_audio(path)
        print("Transcribing with Whisper...")
        result = whisper_model.transcribe(audio_path)
        transcript = result["text"].strip()

        if transcript:
            print(f"Transcription complete! ({len(transcript)} characters)")
            for idx, piece in enumerate(chunk_text(transcript)):
```

```python
        if transcript:
            print(f"Transcription complete! ({len(transcript)} characters)")
            for idx, piece in enumerate(chunk_text(transcript)):
                chunks.append({
                    "id": f"video_audio_{idx}",
                    "text": piece,
                    "type": "video_transcript",
                    "page": 0
                })

    # ── Frames → Vision LLM ──
    print("Extracting frames...")
    frames = extract_frames(path, interval=30)  # 1 frame every 30 seconds
    print(f"Found {len(frames)} frames → Vision LLM")

    for frame_idx, frame_path in enumerate(frames):
        text = vision_extract(frame_path).strip()
        if text:
            for idx, piece in enumerate(chunk_text(text)):
                chunks.append({
                    "id": f"video_frame_{frame_idx}_{idx}",
                    "text": piece,
                    "type": "video_frame",
                    "page": 0
                })

    return chunks
```

Figure 6.5 Ingestion Pipeline

➔ PDF Processing Pipeline

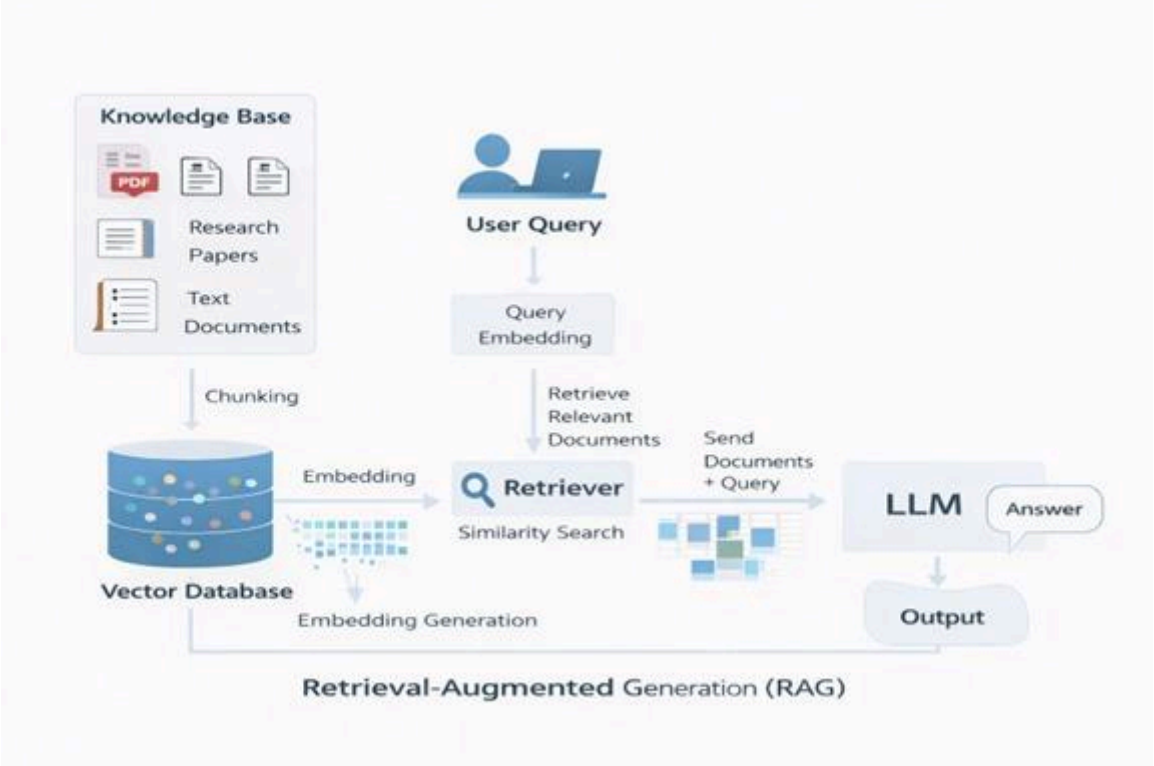


Figure 6.6 Document Pipeline

When a PDF file is uploaded, the system first attempts to extract structured text using the Unstructured library.

chunks = structured_pdf_ingest(path)

The partition_pdf() function analyzes the document and extracts structured elements such as:

- Titles
- Paragraphs
- Lists
- Sections

Each extracted element is stored with metadata such as page number and content type.

➔ **Embedded Image Extraction**

Some PDFs contain diagrams, charts, or screenshots embedded inside the document. These images are extracted using PyMuPDF.

image_chunks = extract_images_from_pdf(path)

The extracted images are processed by the Vision LLM to generate textual descriptions. These descriptions are then added to the existing text chunks.

This step allows the system to understand visual content such as flowcharts and diagrams.

➔ **Image Processing Pipeline**

If the input file is an image (.jpg, .png, .jpeg), the system processes it directly using the Vision Language Model.

The image is first converted into Base64 format and sent to the Vision LLM through an API request.

The Vision model analyzes the image and generates a detailed description including:

- text present in the image
- diagram explanations
- relationships between visual elements

The extracted text is then divided into smaller chunks for further processing.

➔ **Video Processing Pipeline**

When a video file is uploaded, the system extracts both audio information and visual content.

Audio Processing

The system first extracts the audio track from the video using FFmpeg.

audio_path = extract_audio(path)

The audio is then transcribed into text using the Whisper speech recognition mode

result = whisper_model.transcribe(audio_path)

This transcription captures spoken information contained in the video.

➔ **Frame Extraction**

In addition to audio transcription, the system extracts frames from the video at fixed intervals.

frames = extract_frames(path, interval=30)

Each frame is processed using the Vision Language Model to extract textual descriptions of visual content appearing in the video.

2. **Text Chunking**

After extracting text from PDFs, images, and videos, the system divides the content into smaller segments using a chunking mechanism.

def chunk_text(text, chunk_size=400, overlap=80)

Chunking improves retrieval performance by ensuring that each segment contains meaningful contextual information.

3. **Embedding Generation and Vector Storage Module**

After the Data Ingestion Module extracts textual content from PDFs, images, and videos, the next step is to convert this textual data into numerical vector representations. This process is handled by the Embedding Generation and Vector Storage Module.

```python
from sentence_transformers import SentenceTransformer
import chromadb
import shutil
import os

model = SentenceTransformer("all-MiniLM-L6-v2")

# ✅ Clear old DB every time
if os.path.exists("./chroma_db"):
    shutil.rmtree("./chroma_db")
    print("Old DB cleared!")

client = chromadb.PersistentClient(path="./chroma_db")

texts = [item["text"] for item in chunked]
ids = [item["id"] for item in chunked]
metas = [{"page": item["page"] if item["page"] else 0, "type": item["type"]} for item in chunked]

embeddings = model.encode(
    texts,
    normalize_embeddings=True
).tolist()

collection = client.get_or_create_collection("pdf_chunks")

collection.add(
    ids=ids,
    embeddings=embeddings,
    documents=texts,
    metadatas=metas
)
```

Figure 6.7 Embedding Layer

The purpose of this module is to transform textual chunks into semantic embeddings and store them in a vector database so that relevant information can be efficiently retrieved during query processing.

➔ **Text Chunk Collection**

The ingestion module produces structured text chunks containing extracted information from documents and multimedia inputs. Each chunk contains the following information:

- Text content
- Unique chunk identifier
- Metadata such as page number and content type

Example structure of a chunk:

```
collection.add(
    ids=ids,
    embeddings=embeddings,
    documents=texts,
    metadatas=metas
)
```

Figure 6.8 Chunking structure

These chunks are stored in a list called chunked, which is used as input for embedding generation.

➔ **Embedding Generation**

To convert textual chunks into vector representations, the system uses the SentenceTransformer embedding model.

model = SentenceTransformer("all-MiniLM-L6-v2")

The all-MiniLM-L6-v2 model is a lightweight transformer-based model designed to generate high-quality sentence embeddings. These embeddings capture the semantic meaning of text, enabling similarity-based retrieval.

Each text chunk is converted into a vector using the following operation:

embeddings = model.encode(texts, normalize_embeddings=True)

Normalization ensures that the embeddings are scaled properly, which improves similarity search performance in the vector database.

➔ **Preparing Data for Storage**

Before storing the embeddings, the system prepares three components:

Text Content

texts = [item["text"] for item in chunked]

This list contains the extracted text segments.

Unique Identifiers

```
ids = [item["id"] for item in chunked]
```

Each chunk is assigned a unique identifier for indexing in the database.

Metadata

```
metas = [{"page": item["page"], "type": item["type"]} for item in chunked]
```

Metadata provides additional context such as: Page number, type of content (text, image description, video transcript)

This metadata is useful during retrieval to identify the source of the information.

➔ **Vector Database Initialization**

The system uses ChromaDB as the vector database to store embeddings.

First, any existing database is cleared to ensure a fresh storage environment.

```
if os.path.exists("./chroma_db"):
    shutil.rmtree("./chroma_db")
```

Then a persistent client is created:

```
client = chromadb.PersistentClient(path="./chroma_db")
```

Persistent storage allows embeddings to be saved locally so they can be reused without recomputing them.

➔ **Creating a Collection**

In ChromaDB, embeddings are stored inside a **collection**. A collection acts as a container that stores vectors and associated metadata.

```
collection = client.get_or_create_collection("pdf_chunks")
```

This collection stores all processed chunks from the document.

➔ **Storing Embeddings in the Vector Database**

Finally, the embeddings are added to the collection along with their corresponding text and metadata.

Each stored record contains the embedding vector, the original text chunk, metadata information

This structure allows the system to retrieve both the vector representation and the original text during search.

4. Query Processing

After the document data is processed and stored in the vector database (ChromaDB), the system handles user queries through a retrieval and generation pipeline. This pipeline retrieves relevant document content and uses a Large Language Model (LLM) to generate an answer based on the retrieved context.

The query processing pipeline consists of several stages including query expansion, query type detection, hybrid retrieval, prompt construction, and answer generation.

➔ **User Query**

The process begins when a user submits a question to the system. The query represents the information the user intends to retrieve from the processed documents.

Example query:

```
print(aske("What is the proposed system about?"))
```

Figure 6.9 Example query

The query is passed to the main query handling function:

aske(query)

This function orchestrates the retrieval and answer generation process.

```
def aske(query):
    query = expand_query(query)  # ✅ add this line

    title_keywords = ["title", "name of this", "what is this document", "document called"]
    is_title_query = any(kw in query.lower() for kw in title_keywords)

    if is_title_query:
        retrieved = retrieve(query, k=3)
    else:
        retrieved = hybrid_retrieve(query, k=5)

    return generate_answer(query, retrieved)
```

Figure 6.10 Query function

➔ Query Expansion

Before retrieval, the system performs query expansion to improve retrieval accuracy. Query expansion helps the system understand abbreviations or short forms that may appear in the query.

For example:

OCR → Optical Character Recognition

RAG → Retrieval Augmented Generation

NLP → Natural Language Processing

This step ensures that the query terms better match the terminology used in the document.

The query expansion step is implemented as follows:

query = expand_query(query)

By expanding abbreviations, the system improves semantic matching between the query and the document chunks stored in the vector database.

```
def expand_query(query):
    response = client_llm.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=[
            {
                "role": "system",
                "content": """You are a query expansion assistant.
ONLY expand clear abbreviations like CNN, RNN, NLP, RAG, OCR, ML, AI, DL etc.
If the query has NO abbreviations, return it EXACTLY as is without any changes.
Return ONLY the query, nothing else."""
            },
            {
                "role": "user",
                "content": f"Expand abbreviations if any: {query}"
            }
        ],
        temperature=0
    )
    expanded = response.choices[0].message.content.strip()

    if len(expanded) > len(query) * 2:
        return query

    return expanded
```

Figure 6.11 Query expansion layer

➔ **Query Type Detection**

The system also checks whether the query is related to the document title. Title-related queries require special handling because the title typically appears only once in the document.

To detect title queries, the system searches for specific keywords such as:

title_keywords = ["title", "name of this", "what is this document", "document called"]

If the query contains any of these keywords, it is classified as a title query.

Example:

“What is the title of this document?”

When a title query is detected, the system retrieves chunks that are specifically labeled as main_title during document ingestion.

5. Hybrid Retrieval Module

After generating embeddings and storing them in the vector database, the next stage of the system is the Hybrid Retrieval Module. This module is responsible for retrieving the most relevant document chunks from the knowledge base in response to a user query.

The proposed system combines two retrieval approaches:

- ➔ Semantic Vector Retrieval
- ➔ Keyword-Based Retrieval (BM25)

Combining these two methods improves retrieval accuracy because it captures both semantic similarity and exact keyword matching.

➔ Keyword-Based Retrieval (BM25)

The system first builds a keyword search index using the BM25 ranking algorithm. BM25 is a widely used probabilistic information retrieval algorithm that ranks documents based on the frequency and importance of query terms.

The corpus used for keyword retrieval is constructed from the extracted text chunks.

```
from rank_bm25 import BM25Okapi

corpus = [chunk["text"] for chunk in chunked]
tokenized_corpus = [doc.split() for doc in corpus]
bm25 = BM25Okapi(tokenized_corpus)
print("BM25 rebuilt with", len(corpus), "chunks")  # ✅ only added this line
```

```
BM25 rebuilt with 56 chunks
```

Figure 6.12 bm250

Each document chunk is tokenized into individual words, and the BM25 model calculates relevance scores based on term frequency and document length.

```python
def keyword_search(query, k=5):
    tokenized_query = query.split()

    scores = bm25.get_scores(tokenized_query)

    ranked = sorted(
        list(zip(chunked, scores)),
        key=lambda x: x[1],
        reverse=True
    )

    results = []

    for chunk, score in ranked[:k]:
        results.append((chunk["text"], score, chunk))

    return results
```

Figure 6.13 keyword retrieve

When a query is received, the BM25 model calculates a score for each document chunk and returns the top matching results.

➔ **Semantic Vector Retrieval**

In addition to keyword search, the system performs semantic retrieval using vector embeddings stored in the ChromaDB database.

```python
def retrieve(query, k=5):
    q_emb = model.encode(query, normalize_embeddings=True).tolist()

    title_keywords = ["title", "name of this", "what is this document", "document called"]
    is_title_query = any(kw in query.lower() for kw in title_keywords)

    if is_title_query:
        # Try main_title first
        results = collection.query(
            query_embeddings=[q_emb],
            n_results=k,
            where={
                "$and": [
                    {"type": {"$eq": "main_title"}},
                    {"page": {"$eq": 1}}
                ]
            },
            include=["documents", "metadatas", "distances"]
        )
        # Fallback if no main_title found
        if not results["documents"][0]:
            results = collection.query(
                query_embeddings=[q_emb],
                n_results=k,
                include=["documents", "metadatas", "distances"]
            )
    else:
        results = collection.query(
            query_embeddings=[q_emb],
```

```python
                type : { $in : [ NarrativeText , ListItem , ocr_text , image_ocr , video_transcript , video_frame ]}
            },
            include=["documents", "metadatas", "distances"]
        )

    docs = results["documents"][0]
    distances = results["distances"][0]
    metas = results["metadatas"][0]

    candidates = [
        (doc, dist, meta)
        for doc, dist, meta in zip(docs, distances, metas)
        if not any(phrase in doc.lower() for phrase in ["presented by", "submitted by", "guided by"])
    ]

    if not candidates:
        candidates = list(zip(docs, distances, metas))

    candidates.sort(key=lambda x: x[1])
    return candidates


results = retrieve("what are the two main sections in daigram")
for doc, dist, meta in results:
    print(f"Page {meta['page']} | Score: {dist:.3f} | {doc[:80]}")
```

Figure 6.14 Semantic retrieve layer

First, the user query is converted into an embedding vector using the SentenceTransformer model.

q_emb = model.encode(query, normalize_embeddings=True)

The system then performs a similarity search in the vector database to retrieve the most semantically relevant document chunks.

collection.query(

query_embeddings=[q_emb],

n_results=k

)

Vector similarity search enables the system to retrieve information based on semantic meaning rather than exact word matches.

➔ **Hybrid Retrieval Strategy**

The final retrieval results are obtained by combining both vector search results and BM25 keyword search results.

```
def hybrid_retrieve(query, k=5):
    vector_results = retrieve(query, k*2)
    keyword_results = keyword_search(query, k*2)

    max_vdist = max([s for _, s, _ in vector_results]) if vector_results else 1
    if max_vdist == 0: max_vdist = 1

    vector_scored = {
        doc[:100]: {"doc": doc, "meta": meta, "score": 1 - (dist / max_vdist)}
        for doc, dist, meta in vector_results
    }

    max_bm25 = max([s for _, s, _ in keyword_results]) if keyword_results else 1
    if max_bm25 == 0: max_bm25 = 1

    keyword_scored = {
        doc[:100]: {"doc": doc, "meta": meta, "score": score / max_bm25}
        for doc, score, meta in keyword_results
    }

    all_keys = set(vector_scored) | set(keyword_scored)
    merged = []

    for key in all_keys:
        v_score = vector_scored.get(key, {}).get("score", 0)
        k_score = keyword_scored.get(key, {}).get("score", 0)
        final_score = (0.7 * v_score) + (0.3 * k_score)  # ✅ more vector weight
```

```
        k_score = keyword_scored.get(key, {}).get("score", 0)
        final_score = (0.7 * v_score) + (0.3 * k_score)  # ✅ more vector weight

        # ✅ filter weak matches
        if final_score < 0.1:
            continue

        doc_info = vector_scored.get(key) or keyword_scored.get(key)
        merged.append((doc_info["doc"], final_score, doc_info["meta"]))

    merged.sort(key=lambda x: x[1], reverse=True)
    return merged[:k]
```

Figure 6.15 Hybrid retrieve layer

The system first retrieves results from both methods:

vector_results = retrieve(query, k*2)

keyword_results = keyword_search(query, k*2)

Each result is assigned a normalized score. The final ranking score is calculated using a weighted combination of the two retrieval methods.

final_score = (0.7 * v_score) + (0.3 * k_score)

In this system:

- 70% weight is assigned to semantic vector similarity
- 30% weight is assigned to keyword-based BM25 search

This weighting prioritizes semantic understanding while still benefiting from keyword matching.

Low-quality matches are filtered using a minimum score threshold to remove weak results.

if final_score < 0.1:

continue

The results are then sorted based on the final score, and the top-k relevant document chunks are returned.

7. LLM Answer Generation Module

After retrieving the most relevant document chunks using the Hybrid Retrieval Module, the next stage of the system is the LLM Answer Generation Module. This module is responsible for generating a natural language response to the user's query using a Large Language Model (LLM).

The system uses the Llama 3.1 8B Instant model to produce context-aware answers.

The retrieved document chunks serve as contextual information, which is provided to the language model along with the user's query.

```
def generate_answer(query, retrieved_chunks):
    context = "\n\n".join([doc for doc, _, _ in retrieved_chunks])

    prompt = f"""You are a helpful assistant answering questions based on the provided context.

Use the context below to answer the question.
If the context contains relevant information, use it to give a clear answer.
If the context truly does not contain the answer, say "The document does not provide this information."

Context:
{context}

Question:
{query}

Answer:"""

    response = client_llm.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=[
            {"role": "system", "content": "Answer based on the context. Be helpful and extract relevant information even if not explicitly stated."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.3  # slightly higher for better answers
    )

    return response.choices[0].message.content
```

Figure 6.16 Answer generation layer

➔ Context Preparation

The first step in answer generation is to combine the retrieved document chunks into a single context block.

context = "\n\n".join([doc for doc, _, _ in retrieved_chunks])

This ensures that all relevant information retrieved from the knowledge base is available to the language model during response generation.

The combined context represents the knowledge source from which the model generates the answer.

➔ Prompt Construction

A structured prompt is created to guide the language model in generating accurate answers. The prompt includes:

- instructions for the model
- the retrieved context
- the user's question

```
def generate_answer(query, retrieved_chunks):
    context = "\n\n".join([doc for doc, _, _ in retrieved_chunks])

    prompt = f"""You are a helpful assistant answering questions based on the provided context.

Use the context below to answer the question.
If the context contains relevant information, use it to give a clear answer.
If the context truly does not contain the answer, say "The document does not provide this information."

Context:
{context}

Question:
{query}

Answer:"""
```

Figure 6.17 Prompt Construction

This prompt ensures that the model generates answers strictly based on the retrieved context rather than relying on external knowledge.

➔ **Response Generation**

The model analyzes the provided context and generates a response that answers the user's query.

The temperature parameter is set to 0.3, which helps maintain a balance between response accuracy and natural language generation.

Once the prompt is prepared, the system sends the prompt to the Large Language Model.

```
response = client_llm.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "system", "content": "Answer based on the context. Be helpful and extract relevant information even if not explicitly stated."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.3  # slightly higher for better answers
)

return response.choices[0].message.content
```

Figure 6.18 Response layer

6.3 Training

In traditional machine learning systems, the dataset is divided into training and testing sets to train a model and evaluate its performance. However, the proposed system follows a Retrieval-Augmented Generation (RAG) architecture, which relies on pretrained models rather than training a model from scratch.

Therefore, instead of a conventional training process, the system focuses on building a knowledge base from input documents and evaluating the retrieval and generation performance using test queries.

Knowledge Base Preparation

In this system, the training phase refers to the process of preparing the knowledge base used by the RAG pipeline.

It consists of the following steps:

➔ Document Collection

Input documents such as PDFs, images, and videos are collected and uploaded to the system.

➔ Data Ingestion

The Data Ingestion Module processes these files and extracts textual information from:

- PDF documents
- Images and diagrams
- Video transcripts and frames

➔ Text Chunking

Extracted text is divided into smaller segments called chunks to improve retrieval accuracy.

➔ Embedding Generation

Each chunk is converted into a vector representation using the SentenceTransformer embedding model (all-MiniLM-L6-v2).

➔ Vector Database Storage

The embeddings are stored in ChromaDB, forming a searchable knowledge base.

This stage effectively prepares the system for answering queries by creating a structured vector representation of the input documents.

6.4 Testing Phase

The testing phase evaluates the system's ability to retrieve relevant information and generate accurate answers.

During testing, a set of predefined test queries is used to interact with the system.

Example queries include:

- What is the proposed system about?
- What technologies are used?
- What does the embedding model do?
- Who presented the project?

For each query, the system performs the following operations:

- Query Expansion (optional abbreviation expansion)
- Hybrid Retrieval
    - ➔ Vector similarity search using embeddings
    - ➔ Keyword-based retrieval using BM25

- Context Retrieval

The most relevant document chunks are selected.

- Answer Generation

The retrieved context is passed to the Large Language Model to generate the final answer.

7. Results and Discussion

To assess the performance of the proposed Retrieval-Augmented Generation (RAG) system, multiple evaluation metrics are used to measure the effectiveness of different stages of the pipeline. Since the system consists of retrieval and answer generation components, evaluation must consider both the quality of retrieved context and the quality of generated answers.

The evaluation strategy is divided into four main categories:

- Retrieval Performance
- Context Quality
- Answer Quality
- Text Similarity Metrics

These metrics help determine how accurately the system retrieves relevant information and how effectively the language model generates responses based on the retrieved context.

➔ **Retrieval Performance**

- Recall@k Evaluation

To evaluate the effectiveness of the retrieval module, the system uses Recall@k, which measures whether the relevant information appears within the top k retrieved document chunks.

Recall@k is calculated using the following formula:

Recall@k = (Number of queries where relevant information is retrieved in top k) / (Total number of queries)

Importance of Recall@k

Recall@k is an important metric in Retrieval-Augmented Generation systems because the language model relies on retrieved context to generate answers.

If the relevant document chunk is not retrieved, the language model will not have access to the correct information, which may result in inaccurate answers.

Therefore, improving Recall@k directly improves the overall performance of the question answering system.

```
def evaluate_retrieval(test_set):
    correct = 0

    for query, keyword in test_set:
        results = hybrid_retrieve(query, k=5)

        retrieved_text = " ".join([doc.lower() for doc,_,_ in results])

        if keyword.lower() in retrieved_text:
            correct += 1

    recall = correct / len(test_set)

    print("Recall@5:", recall)


# Run evaluation
evaluate_retrieval(test_set)
```

```
Recall@5: 0.5
```

Figure 7.1 Recal@K

This means that 50% of the evaluated queries successfully retrieved relevant information within the top five document chunks.

A Recall@5 score of 0.5 indicates that the retrieval system is able to retrieve relevant information for half of the tested queries.

This suggests that:

The hybrid retrieval mechanism is capable of identifying relevant document chunks for some queries.

In other words, the relevant information exists in the document corpus but is not always retrieved within the top five results.

➔ **Answer Generation Evaluation**

1. **LLM-Based Evaluation of Generated Answers**

To evaluate the quality of answers produced by the proposed Retrieval-Augmented Generation (RAG) system, an LLM-based evaluation approach is used. Instead of relying only on traditional text similarity metrics, a large language model is used as an automated judge to assess whether the generated answers are accurate and relevant.

This evaluation focuses on two important criteria:

- Faithfulness
- Relevance

The evaluation is implemented using the Llama 3.1 8B Instant model.

Evaluation Criteria

Faithfulness

Faithfulness measures whether the generated answer is grounded in the retrieved document context.

A faithful answer must rely only on information present in the retrieved document chunks. If the model introduces information that does not appear in the retrieved context, the answer is considered unfaithful.

Evaluation Method

For each query, the following inputs are provided to the evaluation model:

- the user query
- the retrieved context
- the generated answer

The language model analyzes these inputs and returns a JSON response containing the evaluation results.

prompt used for evaluation:

```
{
  "faithful": true/false,
  "relevant": true/false
}
```

This allows the system to automatically determine whether the generated answer is correct and contextually grounded.

Evaluation Process

The evaluation is conducted using the evaluate_rag() function.

For each query in the test set, the system performs the following steps:

- Retrieve the most relevant document chunks using hybrid retrieval.
- Combine the retrieved chunks into a single context block.
- Generate an answer using the RAG pipeline.
- Send the query, retrieved context, and generated answer to the LLM judge.
- Record whether the answer is faithful and relevant.

The results are accumulated across all queries to calculate overall evaluation scores.

Example Evaluation Queries

The evaluation was performed using several representative queries, including:

- What is the title of the document?
- What problem do traditional chatbots have?
- What is the proposed system about?
- What pipeline does the system follow?

- What technologies are used?
- What does ChromaDB do?

These queries test the system's ability to retrieve information and generate accurate answers from the document.

```
def llm_judge(query, answer, context):
    """Use LLM to judge if answer is faithful to context"""
    prompt = f"""You are an evaluation judge for a RAG system.

Question: {query}
Retrieved Context: {context[:500]}
Generated Answer: {answer}

Evaluate on two criteria. Respond in JSON only:
{{
  "faithful": true/false,  // Is the answer grounded in the context?
  "relevant": true/false   // Does the answer address the question?
}}"""

    response = client_llm.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )

    import json
    try:
        return json.loads(response.choices[0].message.content)
    except:
        return {"faithful": False, "relevant": False}
```

```python
def evaluate_rag(queries):
    faithful_scores = []
    relevant_scores = []

    for query in queries:
        # Get answer and context
        results = hybrid_retrieve(query, k=5)
        context = " ".join([doc for doc, _, _ in results])
        answer = aske(query)

        # LLM judges
        scores = llm_judge(query, answer, context)
        faithful_scores.append(scores["faithful"])
        relevant_scores.append(scores["relevant"])

        status_f = "✅" if scores["faithful"] else "❌"
        status_r = "✅" if scores["relevant"] else "❌"
        print(f"{status_f} Faithful | {status_r} Relevant | {query}")

    faithfulness = sum(faithful_scores) / len(faithful_scores)
    relevancy = sum(relevant_scores) / len(relevant_scores)

    print(f"\n=== EVALUATION RESULTS ===")
    print(f"Faithfulness : {faithfulness:.2f}")
    print(f"Relevancy    : {relevancy:.2f}")
    print(f"Overall      : {(faithfulness + relevancy) / 2:.2f}")
```

```python
# Test queries — no expected answers needed!
queries = [
    "What is the title of the document?",
    "What problem do traditional chatbots have?",
    "What is the proposed system about?",
    "What pipeline does the system follow?",
    "What technologies are used?",
    "What does ChromaDB do?",
    "What does the embedding model do?",
    "What is the role of the speech-to-text model?",
    "Who presented the project?",
    "What are the limitations of existing systems?"
]

evaluate_rag(queries)
```

Figure 7.2 LLM judging

```
✅ Faithful | ✅ Relevant | What is the title of the document?
✅ Faithful | ✅ Relevant | What problem do traditional chatbots have?
✅ Faithful | ✅ Relevant | What is the proposed system about?
✅ Faithful | ✅ Relevant | What pipeline does the system follow?
✅ Faithful | ✅ Relevant | What technologies are used?
✅ Faithful | ✅ Relevant | What does ChromaDB do?
✅ Faithful | ✅ Relevant | What does the embedding model do?
❌ Faithful | ❌ Relevant | What is the role of the speech-to-text model?
❌ Faithful | ❌ Relevant | Who presented the project?
❌ Faithful | ❌ Relevant | What are the limitations of existing systems?

=== EVALUATION RESULTS ===
Faithfulness : 0.70
Relevancy    : 0.70
Overall      : 0.70
```

Figure 7.3 Judging Output

This evaluation determines whether the generated answers are accurately grounded in the retrieved document context and whether they correctly address the user's query.

This means that 70% of the generated responses were both faithful to the retrieved context and relevant to the user's query.

Understanding Faithfulness

- In other words, it checks whether the language model:
- uses information from the retrieved document chunks
- avoids introducing information that is not present in the context
- does not hallucinate unsupported facts.

Faithfulness measures whether the generated answer is supported by the retrieved context.

A faithfulness score of 0.70 indicates that the system produced contextually grounded responses for 7 out of 10 queries.

Understanding Relevance

Relevance measures whether the generated answer correctly addresses the user's question.

An answer is considered relevant if it:

- directly answers the question

- contains the correct information required by the query
- maintains logical consistency with the question.

The obtained relevance score of 0.70 indicates that 7 out of the 10 generated responses effectively addressed the user's queries.

2. Text Similarity Evaluation Using BLEU and ROUGE

In addition to retrieval and LLM-based evaluation, the proposed system is also evaluated using text similarity metrics. These metrics compare the generated answers with reference answers derived from the source document. The objective of this evaluation is to measure how closely the generated responses match the expected answers.

Two widely used natural language generation metrics are used in this project:

- BLEU Score(Bilingual Evaluation Understudy)
- ROUGE Score

These metrics help quantify the similarity between the generated answer and the ground truth reference answer.

➔ BLEU Score

BLEU (Bilingual Evaluation Understudy) is a metric commonly used to evaluate machine-generated text by comparing it with a reference text. It measures the overlap of words and phrases between the generated response and the reference answer.

```
!pip install rouge-score nltk

import nltk
nltk.download('punkt')
nltk.download('punkt_tab')

from rouge_score import rouge_scorer
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
import numpy as np

# Reference answers from your document
qna_pairs = [
    ("What is the proposed system about?",
     "The proposed system is an intelligent Question Answering chatbot that responds to user queries based on user-uploaded documents and media using RAG pipeline"),
    ("What problem do traditional chatbots have?",
     "Traditional chatbots cannot process diagrams images or video and have high risk of hallucination"),
    ("What pipeline does the system follow?",
     "The system follows the Retrieval-Augmented Generation RAG pipeline"),
    ("What does ChromaDB do?",
     "ChromaDB is used for fast vector similarity search"),
    ("What does the embedding model do?",
     "The embedding model converts text from all sources into numerical vectors to capture semantic meaning"),
]

scorer = rouge_scorer.RougeScorer(['rouge1', 'rougeL'], use_stemmer=True)
smoothie = SmoothingFunction().method4
```

```
print("=== BLEU / ROUGE SCORES ===\n")

for query, reference in qna_pairs:
    generated = aske(query)

    # ROUGE
    scores = scorer.score(reference, generated)
    rouge1_scores.append(scores['rouge1'].fmeasure)
    rougeL_scores.append(scores['rougeL'].fmeasure)

    # BLEU
    ref_tokens = [reference.lower().split()]
    gen_tokens = generated.lower().split()
    bleu = sentence_bleu(ref_tokens, gen_tokens, smoothing_function=smoothie)
    bleu_scores.append(bleu)

    print(f"Q: {query}")
    print(f"BLEU: {bleu:.2f} | ROUGE-1: {scores['rouge1'].fmeasure:.2f} | ROUGE-L: {scores['rougeL'].fmeasure:.2f}")
    print()

print("=== AVERAGE SCORES ===")
print(f"BLEU    : {np.mean(bleu_scores):.2f}")
print(f"ROUGE-1: {np.mean(rouge1_scores):.2f}")
print(f"ROUGE-L: {np.mean(rougeL_scores):.2f}")
```

Figure 7.4 BLEU

In this project, BLEU score is calculated using the NLTK library.

The generated answer is tokenized into words and compared with the reference answer tokens. The BLEU score ranges from 0 to 1, where:

- 1 indicates a perfect match
- 0 indicates no similarity

Because generated answers may use different wording while still conveying the same meaning, smoothing techniques are applied to improve BLEU score calculation.

➔ ROUGE Score

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures the overlap between the generated text and the reference answer. It is commonly used for evaluating summarization and question answering systems.

In this project, the rouge-score library is used to calculate ROUGE metrics.

ROUGE-1

Measures the overlap of individual words (unigrams) between the generated answer and the reference answer.

ROUGE-L

Measures the longest common subsequence between the generated and reference text, capturing structural similarity between sentences.

Both scores range from 0 to 1, where higher values indicate greater similarity.

```
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nlt
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
=== BLEU / ROUGE SCORES ===

Q: What is the proposed system about?
BLEU: 0.81 | ROUGE-1: 0.93 | ROUGE-L: 0.93

Q: What problem do traditional chatbots have?
BLEU: 0.02 | ROUGE-1: 0.13 | ROUGE-L: 0.10

Q: What pipeline does the system follow?
BLEU: 0.68 | ROUGE-1: 1.00 | ROUGE-L: 1.00

Q: What does ChromaDB do?
BLEU: 0.03 | ROUGE-1: 0.39 | ROUGE-L: 0.33

Q: What does the embedding model do?
BLEU: 0.04 | ROUGE-1: 0.24 | ROUGE-L: 0.21

=== AVERAGE SCORES ===
BLEU   : 0.32
ROUGE-1: 0.54
ROUGE-L: 0.52
```

Figure 7.5 output of bleu and rouge

The system captures most important information from the document.

Generated answers are often paraphrased, which lowers BLEU but still keeps good ROUGE.

Query-wise Analysis

```
Q: What is the proposed system about?
BLEU: 0.81 | ROUGE-1: 0.93 | ROUGE-L: 0.93
```

Figure 7.6 evaluation query 1

These values represent very high similarity between the generated answer and the reference answer, indicating that the system successfully retrieved the correct information and generated an accurate response.

The ROUGE-1 score for this query is 0.93, which indicates that 93% of the important words in the reference answer are also present in the generated answer.

ROUGE-1 measures the overlap of individual words (unigrams) between the generated response and the reference answer. A score close to 1.0 suggests that the generated answer captures nearly all the essential terms used in the reference.

The ROUGE-L score for this query is 0.93, which indicates a strong similarity in sentence structure and sequence of information between the generated answer and the reference answer.

ROUGE-L evaluates the Longest Common Subsequence (LCS) between two sentences. This means it checks how much of the sentence structure and word ordering matches between the generated output and the reference.

```
Q: What problem do traditional chatbots have?
BLEU: 0.02 | ROUGE-1: 0.13 | ROUGE-L: 0.10
```

Figure 7.7 evaluation query 2

These scores are relatively low compared to other queries evaluated in the system. However, this result does not necessarily indicate incorrect answer generation. Instead, the low values primarily arise due to semantic paraphrasing and differences in lexical overlap between the generated response and the reference answer.

The BLEU score obtained for this query is 0.02, which indicates very low n-gram overlap between the generated answer and the reference answer.

BLEU evaluates how closely the generated text matches the exact wording of the reference text by measuring the overlap of n-grams (word sequences).

Reference Answer:

“Traditional chatbots cannot process diagrams, images, or video.”

Generated Answer:

“Traditional chatbots have limitations because they cannot interpret visual information such as diagrams, images, or multimedia content.”

Although both answers express the same meaning, the wording and sentence structure differ significantly. Because BLEU relies on exact word matches, the similarity score becomes very low.

Therefore, the low BLEU score does not necessarily reflect poor system performance, but rather indicates differences in lexical expression.

- ➔ **Context Precision@5 Evaluation**

To evaluate the quality of the retrieved context, the system was tested using the Context Precision@5 metric. This metric measures how many of the retrieved document chunks are actually relevant to the user's query.

```python
def evaluate_context_precision(queries):
    """Check if retrieved chunks are actually relevant to the query"""
    precision_scores = []

    for query in queries:
        results = hybrid_retrieve(query, k=5)
        relevant_count = 0

        for doc, dist, meta in results:
            # Ask LLM if this chunk is relevant to the query
            prompt = f"""Is this context chunk relevant to answer the query?
Query: {query}
Chunk: {doc[:200]}
Reply with only YES or NO."""

            response = client_llm.chat.completions.create(
                model="llama-3.1-8b-instant",
                messages=[{"role": "user", "content": prompt}],
                temperature=0
            )

            answer = response.choices[0].message.content.strip().upper()
            if "YES" in answer:
                relevant_count += 1

        precision = relevant_count / 5  # k=5
        precision_scores.append(precision)
```

```python
        precision = relevant_count / 5  # k=5
        precision_scores.append(precision)
        print(f"Precision: {precision:.2f} | {query}")

    avg = sum(precision_scores) / len(precision_scores)
    print(f"\nContext Precision@5: {avg:.2f}")

# Questions your document ACTUALLY answers well
queries = [
    "What is the proposed system about?",
    "What problem do traditional chatbots have?",
    "What are the limitations of existing chatbot systems?",
    "What is the title of this document",
    "What does the OCR model do?",
]

evaluate_context_precision(queries)
```

Figure 7.8 Context precision

In Retrieval-Augmented Generation (RAG) systems, the accuracy of the generated answer largely depends on the quality of the retrieved context. If irrelevant chunks are retrieved, the language model may produce incorrect or incomplete answers. Therefore, evaluating retrieval precision is an important step in assessing the overall performance of the system.

Purpose of Context Precision@5

Context Precision@5 evaluates the relevance of the top 5 retrieved chunks returned by the hybrid retrieval system.

The metric answers the following question:

> Out of the top 5 retrieved chunks, how many actually contain information useful for answering the query?

The formula used for calculating Context Precision@5 is:

$$Precision@5 = \frac{\text{Number of relevant retrieved chunks}}{5}$$

Figure 7.9 Precision equation

```
Precision: 0.60 | What is the proposed system about?
Precision: 0.60 | What problem do traditional chatbots have?
Precision: 0.60 | What are the limitations of existing chatbot systems?
Precision: 0.40 | What does the OCR model do?

Context Precision@5: 0.55
```

Figure 6.24 output of precision

The retrieval component of the proposed Retrieval-Augmented Generation (RAG) system was evaluated using the Context Precision@5 metric. This metric measures the proportion of relevant document chunks among the top five retrieved results returned by the hybrid retrieval system.

The evaluation was conducted on four representative queries related to the project document.

Query	Precision@5
What is the proposed system about?	0.60
What problem do traditional chatbots have?	0.60
What are the limitations of existing chatbot systems?	0.60
What does the OCR model do?	0.40

The average Context Precision@5 score of 0.55 indicates that approximately 55% of the retrieved document chunks are relevant to the user queries.

Since five chunks are retrieved for each query, this means that around 2 to 3 chunks among the retrieved results contain useful information for answering the query.

This result demonstrates that the hybrid retrieval system is able to identify relevant context for most queries, although some irrelevant or partially relevant chunks are still retrieved.

Overall, a precision score above 0.5 is considered acceptable for a small-scale RAG system, especially when working with a limited document dataset.

7.1 CONCLUSION OF RESULTS

- **Retrieval Performance**

The retrieval component of the system was evaluated using Recall@5 and Context Precision@5 metrics.

Recall@5

Recall@5 measures the ability of the retrieval system to retrieve relevant document chunks within the top five retrieved results.

The system achieved:

Recall@5 = 0.50

This indicates that the system successfully retrieved relevant information for 50% of the evaluated queries within the top five retrieved document chunks.

This result suggests that the hybrid retrieval strategy combining vector similarity search and BM25 keyword search is capable of identifying relevant context for several queries. However, in some cases the relevant information was not retrieved within the top five results. This may occur due to:

- limitations of the embedding model in capturing semantic similarity,
- variations between query phrasing and document wording,
- chunking strategies that split relevant information across multiple segments.

Although the recall value is moderate, the retrieved context often contained partial information that still enabled the language model to generate meaningful answers.

Context Precision@5

Context Precision@5 evaluates the relevance of the retrieved document chunks.

The system achieved:

Context Precision@5 = 0.55

This means that approximately 55% of the retrieved chunks were relevant to the query.

In practical terms, this indicates that for most queries the system retrieved 2–3 relevant chunks among the top five results.

The relatively good precision score demonstrates that the hybrid retrieval approach effectively identifies relevant document segments. However, some irrelevant chunks were also retrieved due to semantic similarities between topics discussed within the document.

For example, chunks discussing related concepts such as embeddings, OCR, and multimodal processing may sometimes be retrieved together even if they are not directly related to the query.

Despite the presence of some irrelevant chunks, the relevant context provided sufficient information for the language model to generate accurate responses.

- **Answer Generation Evaluation**

The quality of generated answers was evaluated using Faithfulness and Relevance metrics, as well as BLEU and ROUGE scores.

Faithfulness and Relevance

Faithfulness measures whether the generated answer is grounded in the retrieved context, while relevance evaluates whether the generated response correctly answers the user's question.

The system achieved:

Faithfulness = 0.70

Relevance = 0.70

Overall score = 0.70

These results indicate that 70% of the generated responses were both contextually grounded and relevant to the query.

The system successfully generated correct answers for most queries

- **BLEU and ROUGE Evaluation**

BLEU and ROUGE metrics were used to measure the similarity between the generated answers and the reference answers extracted from the document

The system achieved the following average scores:

BLEU = 0.32

ROUGE-1 = 0.54

ROUGE-L = 0.52

The ROUGE scores indicate that the generated answers contained more than half of the important words and sentence structures present in the reference answers.

The relatively lower BLEU score is expected because large language models often generate paraphrased responses instead of exact text matches. As a result, even semantically correct answers may have lower BLEU scores due to differences in wording and sentence structure.

For example, the system may produce an expanded explanation rather than copying the exact sentence from the document. Although the meaning remains correct, lexical overlap decreases, which reduces the BLEU score.

7.2 DISCUSSION

The experimental results demonstrate that the proposed RAG-based system is capable of effectively retrieving relevant document information and generating accurate answers for most user queries.

The hybrid retrieval strategy combining semantic vector search and keyword-based retrieval improves the chances of retrieving relevant document chunks. The retrieval results show that the system retrieves relevant context for a significant portion of the queries, although some irrelevant chunks are occasionally included.

The answer generation evaluation indicates that the language model is capable of producing faithful and relevant responses when the correct context is available. The majority of the generated answers were supported by the retrieved document content.

However, several challenges were observed during evaluation:

- Some queries failed due to incomplete or missing retrieval results.
- Certain information appeared only once in the document, reducing the likelihood of retrieval.
- Fixed-size chunking sometimes separated related information across different chunks.

Despite these challenges, the system demonstrated promising performance for a multimodal document question answering system.

Overall, the evaluation results show that the proposed system successfully integrates multimodal document processing, hybrid retrieval, and large language model generation to provide accurate answers based on document content.

8. REFERENCES

- XuZhao, Guozhong Wang*andYufei Lu MDKAG:Retrieval-Augmented Educational QA Powered by a Multimodal Disciplinary Knowledge Graph, *Appl. Sci.* **2025**, *15*(16), 9095; **https://doi.org/10.3390/app15169095**
- Meng, X., Tong, Z. Multimodal retrieval-augmented generation framework for visually rich knowledge in the architecture domain. *ARIN* 4, 22 (2025). https://doi.org/10.1007/s44223-025-00102-6
- Jasper Linders1 · Jakub M. Tomczak, Knowledge graph-extended retrieval augmented generation for question answering, Applied Intelligence (2025) 55:1102 https://doi.org/10.1007/s10489-025-06885-5